

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA SAU ĐẠI HỌC



BÁO CÁO CHUYÊN ĐỀ

**So sánh Bloom Filter và Counting Bloom Filter
trong bài toán lọc dữ liệu tốc độ cao**

Giảng viên: TS. Đỗ Thanh Hà

Học viên thực hiện: NGUYỄN MẠNH CƯỜNG – B25CHHT082
TRẦN DUY KHÁNH – B25CHHT100
NGUYỄN ĐỨC HOÀNG – B25CHHT096

HÀ NỘI – 2026

Tóm tắt nội dung

Báo cáo so sánh Bloom Filter (BF, Bloom 1970) và Counting Bloom Filter (CBF, Fan *et al.* 2000) cho bài toán lọc dữ liệu tốc độ cao (lọc gói tin, deep packet inspection, lọc truy vấn cơ sở dữ liệu, lọc cache, khử trùng lặp). Chúng tôi trình bày cơ sở lý thuyết, công thức xác suất dương tính giả (false positive), phân tích tràn counter, mã giả cho các thao tác insert/query/delete, và một bộ thực nghiệm Python tự cài đặt cả hai cấu trúc. Kết quả trên $n = 100\,000$ khóa xác nhận: (i) tỉ lệ dương tính giả (FPR) thực nghiệm bám sát công thức $p = (1 - e^{-kn/m})^k$ và khi dùng cùng bộ hàm băm, cùng m, k thì BF và CBF cho kết quả truy vấn *trùng nhau từng khóa*; (ii) phần dữ liệu chính (payload) của CBF 4-bit tốn đúng $4 \times$ BF, do đó với cùng ngân sách byte, FPR của CBF (30.1%) cao hơn BF (0.82%) khoảng 37 lần, tức hơn một bậc độ lớn; (iii) CBF hỗ trợ xóa an toàn khi bên gọi (caller) chỉ xóa khóa chắc chắn đã chèn: xóa đúng 20\,000 khóa không làm mất khóa thật nào còn lại, trong khi stress test trên một bộ lọc riêng chứa đủ n khóa, cố ý xóa các false positive, tạo ra 8\,063 false negative (8.06% của n). Kết luận: dùng BF cho tập gần như tĩnh; dùng CBF khi cần xóa động và hệ thống bảo đảm được contract của delete; khi cần xóa với FPR mục tiêu $< 3\%$, Cuckoo Filter là lựa chọn đáng cân nhắc hơn.

1 Giới thiệu

Lọc dữ liệu tốc độ cao quy về bài toán *kiểm tra thành viên tập hợp xấp xỉ* (approximate set-membership): cho tập S và khóa x (gói tin, chữ ký mã độc, khóa cơ sở dữ liệu, fingerprint khối dữ liệu), cần trả lời nhanh “ x chắc chắn không thuộc S ” hoặc “ x có thể thuộc S ” với ngân sách bộ nhớ đủ nhỏ để nằm gọn trong SRAM/cache. Việc chấp nhận một tỉ lệ nhỏ dương tính giả cho phép giảm mạnh không gian lưu trữ so với bảng băm chính xác. Đây là đánh đổi không gian/thời gian kinh điển của Bloom [1].

Các miền ứng dụng tiêu biểu gồm: (i) *lọc gói tin và DPI/IDS*: đối sánh chữ ký Snort bằng Bloom filter song song trên FPGA [8]; (ii) *lọc truy vấn cơ sở dữ liệu* kiểu LSM-tree: RocksDB có thể gắn một BF cho mỗi SSTable để bỏ qua các tệp “chắc chắn không chứa khóa”, tránh I/O đĩa [9]; (iii) *web cache hợp tác*: Cache Digest của Squid và giao thức Summary Cache [2]; (iv) *khử trùng lặp* (deduplication) trên fingerprint khối dữ liệu.

Đóng góp của báo cáo: hệ thống hóa lý thuyết BF/CBF; cài đặt độc lập cả hai cấu trúc (kể cả CBF 4-bit đóng gói nibble đúng nghĩa); và bộ thực nghiệm định lượng bốn khía cạnh: độ chính xác, thông lượng, bộ nhớ, và rủi ro false negative khi xóa.

2 Cơ sở lý thuyết

2.1 Bloom Filter (Bloom, 1970)

BF gồm mảng B có m bit khởi tạo 0 và k hàm băm độc lập, phân bố đều $h_1, \dots, h_k : \mathcal{U} \rightarrow \{0, \dots, m-1\}$. *Insert*(x) đặt $B[h_i(x)] = 1$ với mọi i ; *Query*(x) trả “có thể có” nếu mọi bit tương ứng bằng 1, ngược lại trả “chắc chắn không”. Bit đã bật không bao giờ tự tắt nên BF **không có false negative**, nhưng **có false positive** do va chạm băm. BF **không hỗ trợ xóa**: tắt các bit của x có thể tắt nhầm bit đang được phần tử khác dùng chung.

2.2 Counting Bloom Filter (Fan *et al.*, 2000)

CBF thay mỗi bit bằng một *counter* b bit (thường $b = 4$), ký hiệu $C[i]$. *Insert* tăng, *Delete* giảm k counter tương ứng; *Query* trả “có thể có” khi mọi counter dương. Contract của Delete yêu cầu bên gọi (caller) chỉ xóa khóa chắc chắn đã chèn: câu trả lời “có thể có” của Query không chứng minh membership và không thể dùng làm điều kiện đủ để xóa. Khi contract này được tuân thủ và counter không mất thông tin do tràn, CBF không sinh false negative. Hai nguồn phá vỡ bảo đảm là *tràn counter* khi hơn $2^b - 1$ phần tử băm vào cùng một ô, và *xóa sai* một phần tử chưa từng chèn [2, 3].

2.3 Công thức xác suất

Sau khi chèn n phần tử với k hàm băm, xác suất một bit cụ thể còn 0:

$$p_0 = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}. \quad (1)$$

Xác suất dương tính giả (cả k bit của một khóa lạ đều bằng 1):

$$p \approx \left(1 - e^{-kn/m}\right)^k. \quad (2)$$

Cực tiểu hóa p theo k (tương đương yêu cầu tỉ lệ bit 0 bằng 1/2, tức entropy cực đại) cho số hàm băm tối ưu và bộ nhớ tối ưu theo FPR mục tiêu p :

$$k^* = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}, \quad m = -\frac{n \ln p}{(\ln 2)^2} \iff \frac{m}{n} \approx 1.44 \log_2 \frac{1}{p}. \quad (3)$$

Tại k^* : $p = 2^{-(m/n) \ln 2} \approx 0.6185^{m/n}$. Quy tắc thực dụng: $m/n = 8$ cho FPR $\approx 2\%$; $m/n = 10$ cho FPR $< 1\%$.

Trần counter và vì sao 4 bit là đủ. Số phần tử băm vào một counter tuân theo phân bố nhị thức $B(nk, 1/m) \approx \text{Poisson}(\lambda)$ với $\lambda = kn/m = \ln 2$ ở cấu hình tối ưu. Do đó

$$\Pr[C[i] = 16] \approx \frac{e^{-\ln 2} (\ln 2)^{16}}{16!} \approx 6.78 \times 10^{-17}, \quad (4)$$

và Fan *et al.* chặn trên toàn cục: với $k \leq (\ln 2)m/n$, $\Pr[\max_i C[i] \geq 16] \leq m \cdot 1.37 \times 10^{-15}$ [2]. Vì xác suất này vô cùng nhỏ (“minuscule”), counter 4 bit (đếm 0..15) đủ cho hầu hết ứng dụng. CBF tốn $4 \times$ payload so với BF cùng số ô [3]. Nếu insert bão hòa ở 15 rồi delete vẫn giảm như counter thường, số lần tăng vượt ngưỡng đã bị mất và false negative có thể xuất hiện sau nhiều lần xóa. Chính sách “sticky saturation” (không bao giờ giảm ô đã bão hòa) tránh lỗi này nhưng đổi lại tạo false positive dai dẳng; do đó hệ thống thực tế cần theo dõi overflow hoặc rebuild từ nguồn dữ liệu chuẩn (source of truth).

Hai hệ quả cho việc so sánh. (1) Với cùng m , k và cùng bộ hàm băm, một ô của CBF dương khi và chỉ khi bit tương ứng của BF bằng 1 (khi chỉ chèn, chưa xóa), nên hai cấu trúc trả lời truy vấn *giống hệt nhau từng khóa*; CBF không “chính xác hơn” BF. (2) Với cùng *ngân sách byte*, BF có số ô gấp b lần CBF b -bit, nên FPR của BF luôn thấp hơn hẳn (Mục 6.5).

3 Mã giả

Thuật toán 1 Bloom Filter: Insert và Query

```

1: procedure BF-Insert( $x$ )
2:   for  $i \leftarrow 1$  to  $k$  do
3:      $B[h_i(x)] \leftarrow 1$ 
4:   end for
5: end procedure
6: procedure BF-Query( $x$ )
7:   for  $i \leftarrow 1$  to  $k$  do
8:     if  $B[h_i(x)] = 0$  then
9:       return False ▷ chắc chắn không
10:    end if
11:  end for
12:  return True ▷ có thể có
13: end procedure

```

Thuật toán 2 Counting Bloom Filter: Insert, Query, Delete (insert bão hòa)

```
1: procedure CBF-Insert( $x$ )
2:   for  $i \leftarrow 1$  to  $k$  do
3:     if  $C[h_i(x)] < 2^b - 1$  then
4:        $C[h_i(x)] \leftarrow C[h_i(x)] + 1$ 
5:     end if
6:   end for
7: end procedure
8: procedure CBF-Query( $x$ )
9:   for  $i \leftarrow 1$  to  $k$  do
10:    if  $C[h_i(x)] = 0$  then
11:      return False
12:    end if
13:  end for
14:  return True
15: end procedure
16: procedure CBF-Delete( $x$ )
17:  Tiền điều kiện: caller biết  $x$  đã được chèn
18:  for  $i \leftarrow 1$  to  $k$  do
19:    if  $C[h_i(x)] > 0$  then
20:       $C[h_i(x)] \leftarrow C[h_i(x)] - 1$ 
21:    end if
22:  end for
23: end procedure
```

▷ bão hòa ở $2^b - 1$

4 So sánh định tính

Bảng 1: So sánh Bloom Filter và Counting Bloom Filter

Tiêu chí	Bloom Filter	Counting Bloom Filter
Đơn vị ô	1 bit	counter b bit ($b=4$)
Bộ nhớ payload	m bit	$b \cdot m$ bit ($\sim 4\times$)
Xóa (delete)	Không	Có, với khóa biết chắc đã chèn
False positive	Có, $p \approx (1 - e^{-kn/m})^k$	Có (cùng công thức)
False negative	Không bao giờ	Không nếu dùng đúng; có thể khi tràn/xóa sai
Insert/Query	$O(k)$	$O(k)$
FPR cùng ngân sách byte	Thấp hơn ($b \times$ số ô)	Cao hơn
Song song hóa cập nhật	Ghi 1 bit an toàn	Đọc-sửa-ghi cần đồng bộ
Hợp phần cứng	Rất tốt (SRAM on-chip)	Tốn SRAM hơn (thường ở phần mềm)
Ứng dụng điển hình	SSTable filter, Cache Digest, chữ ký DPI tĩnh, dedup	Router xóa flow, cache động, Summary Cache cục bộ

5 Phương pháp thực nghiệm

Môi trường. Python 3.14.4, CPU Intel(R) Core(TM) i7-10700K CPU @ 3.80GHz; các gói mmh3 5.2.1, bytearray 3.10.1, matplotlib 3.11.1, numpy 2.5.2 và pandas 3.0.5. Toàn bộ mã thực nghiệm nằm trong notebook `thuc_nghiem.ipynb`; chạy hết 61 giây.

Cài đặt. Ba cấu trúc được cài đặt từ đầu (notebook `thuc_nghiem.ipynb`): **BF** dùng mảng bit (`bitarray`); **CBF 4-bit** đóng gói 2 counter/byte (nibble packing), đúng chi phí $4\times$ như phân tích lý thuyết; **CBF 8-bit** dùng counter đúng 8 bit (0..255), chi phí $8\times$. Cả ba dùng chung sơ đồ double hashing Kirsch–Mitzenmacher $g_i(x) = (h_1(x) + i h_2(x)) \bmod m$ với h_1, h_2 là MurmurHash3 hai seed khác nhau [5], nhờ đó BF và CBF thăm *cùng các vị trí* với mỗi khóa, qua đó cô lập đúng biến số cần so sánh. Delete chuẩn giả định caller biết khóa đã chèn và không thêm Query; riêng E4 dùng một hàm có bước kiểm tra Query trước khi xóa (Query guard) để chứng minh guard này vẫn không ngăn được xóa sai khi gặp false positive.

Dữ liệu và tham số. $n = 100\,000$ khóa “có mặt” (P000000000...) được chèn và $n' = 100\,000$ khóa “vắng mặt” (A000000000...) dùng đo FPR; sinh tất định để tái lập được. Tỷ lệ $m/n \in \{4, 6, 8, 10, 12, 16\}$; $k = \text{round}((m/n) \ln 2)$ trừ khi quét k . FPR được báo cùng khoảng tin cậy (KTC) Wilson 95%. Các phép đo throughput lặp 5 lượt, dùng median làm giá trị trung tâm và IQR làm độ phân tán; insert/delete dùng filter mới ở mỗi lượt để bảo đảm trạng thái đầu vào giống nhau.

Bốn thực nghiệm.

- E1** FPR thực nghiệm vs. lý thuyết (kèm KTC 95%) và median/IQR throughput của các thao tác insert, query, delete theo m/n .
- E2** FPR theo $k \in \{1, \dots, 12\}$ tại $m/n = 10$ để kiểm chứng $k^* = (m/n) \ln 2$.
- E3** Cố định payload 125 000 byte, so đánh đổi FPR của BF (1 bit/ô), CBF 4-bit và CBF 8-bit.
- E4** Kiểm tra xóa đúng không làm mất khóa thật còn lại, sau đó stress test việc xóa khóa vắng mặt bị CBF báo nhầm “có”.

Giả thuyết. **H1:** FPR thực nghiệm của BF và CBF (cùng m, k) trùng nhau và gần đường lý thuyết trong sai số lấy mẫu. **H2:** payload CBF 4-bit = $4\times$ BF (8-bit = $8\times$). **H3:** trong cài đặt Python đang xét, throughput BF $\geq$ CBF 4-bit cho các thao tác chung (CBF 8-bit có thể bám sát BF ở query trong biên độ nhiễu). **H4:** FPR giảm mũ theo m/n ; cực tiểu theo k đạt quanh k^* .

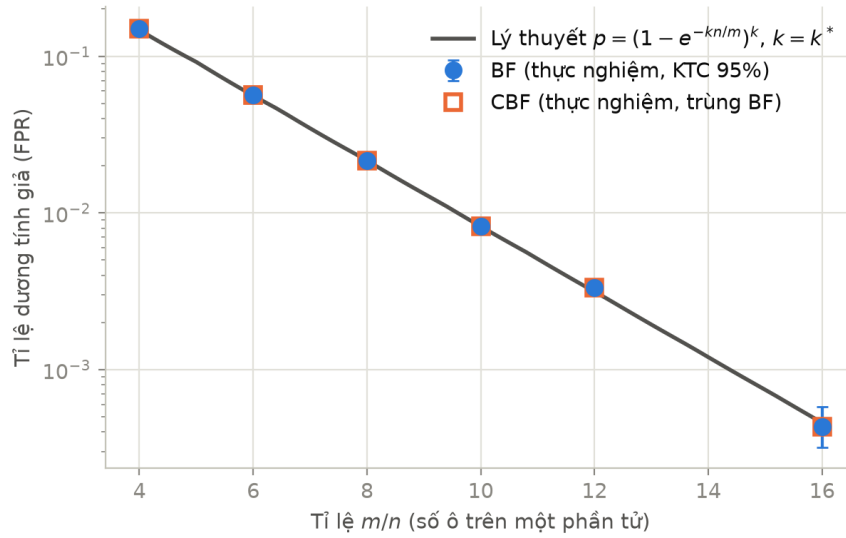
6 Kết quả và thảo luận

6.1 FPR thực nghiệm bám sát lý thuyết (H1, H4)

Bảng 2 và Hình 1 cho thấy FPR thực nghiệm bám sát công thức (2) trên toàn dải m/n : tại $m/n = 8$ lý thuyết 2.16% so với đo được 2.16%; tại $m/n = 10$ lý thuyết 0.82% so với 0.82%, với KTC Wilson 95% [0.765%, 0.877%]. Tỷ lệ bit 1 đo được tại $m/n = 10$ là 0.503 (lý thuyết 0.503, xấp xỉ mức $1/2$ entropy cực đại). Đúng như phân tích ở Mục 2.3, cột FPR của BF và CBF *trùng nhau từng khóa* (được kiểm tra bằng `assert` cho cả CBF 4-bit và 8-bit): khi chỉ chèn, CBF chính là BF nhìn qua phép thử “counter dương”. Khoảng tin cậy chỉ phản ánh sai số trên tập truy vấn vắng mặt cố định; thí nghiệm chưa lấy trung bình qua nhiều họ hash/seed.

Bảng 2: E1: FPR lý thuyết vs. thực nghiệm và KTC Wilson 95% ($n = 100\,000$, $k = \text{round}((m/n) \ln 2)$)

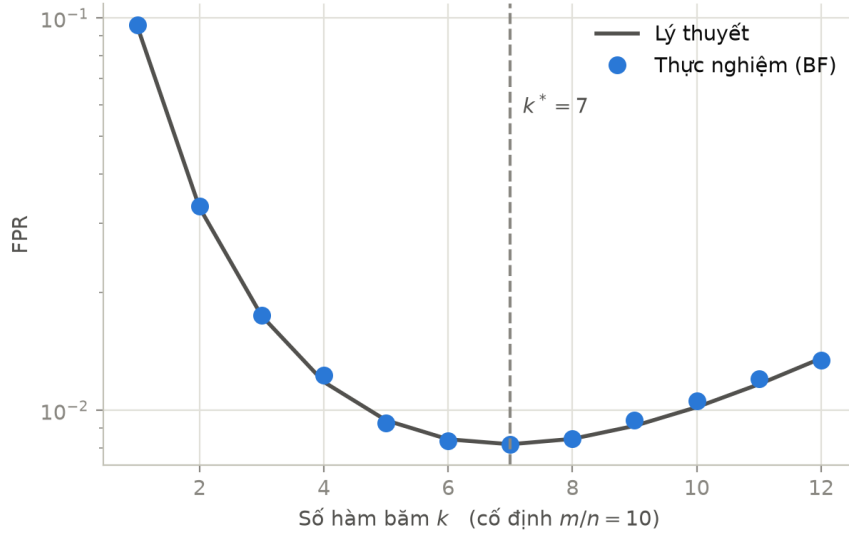
m/n	k	FPR lý thuyết	FPR đo (BF $\equiv$ CBF)	KTC 95%	Tỉ lệ bit 1 (đo)	(lý thuyết)
4	3	14.7%	14.9%	[14.7%, 15.2%]	0.529	0.528
6	4	5.606%	5.650%	[5.509%, 5.795%]	0.487	0.487
8	6	2.158%	2.164%	[2.076%, 2.256%]	0.528	0.528
10	7	0.819%	0.819%	[0.765%, 0.877%]	0.503	0.503
12	8	0.314%	0.334%	[0.300%, 0.372%]	0.487	0.487
16	11	0.046%	0.043%	[0.032%, 0.058%]	0.497	0.497



Hình 1: FPR theo m/n (trục tung log). BF và CBF trùng nhau; thanh sai số là KTC Wilson 95%, đường liền là $p = (1 - e^{-kn/m})^k$ tại $k = k^*$.

6.2 Số hàm băm tối ưu (H4)

Hình 2 quét k từ 1 đến 12 tại $m/n = 10$: FPR đạt cực tiểu quanh $k^* = 7$ đúng như công thức (3). Cơ chế: k quá nhỏ cho một khóa lạ quá ít phép thử độc lập nên hiếm cơ hội bắt gặp được một bit 0; k quá lớn làm đầy mảng bit khiến từng phép thử dễ trùng bit 1 hơn.



Hình 2: FPR theo số hàm băm k tại $m/n = 10$: cực tiểu tại $k^* = (m/n) \ln 2 \approx 7$.

6.3 Bộ nhớ: CBF trả giá $4\times$ (H2)

Bảng 3 xác nhận dung lượng payload của mảng backing: CBF 4-bit (nibble packing) tốn đúng $4.0\times$ BF ở mọi cấu hình; CBF 8-bit tốn $8\times$. Tại $m/n = 10$: BF dùng 125 000 byte, CBF 4-bit 500 000 byte, CBF 8-bit 1 000 000 byte. Các số này không bao gồm object header, allocator hay RSS của tiến trình; hệ số $4\times/8\times$ được suy ra trực tiếp từ cách biểu diễn, không phải một phát hiện về runtime.

Bảng 3: E1: Dung lượng payload của mảng backing theo m/n ($n = 100\,000$)

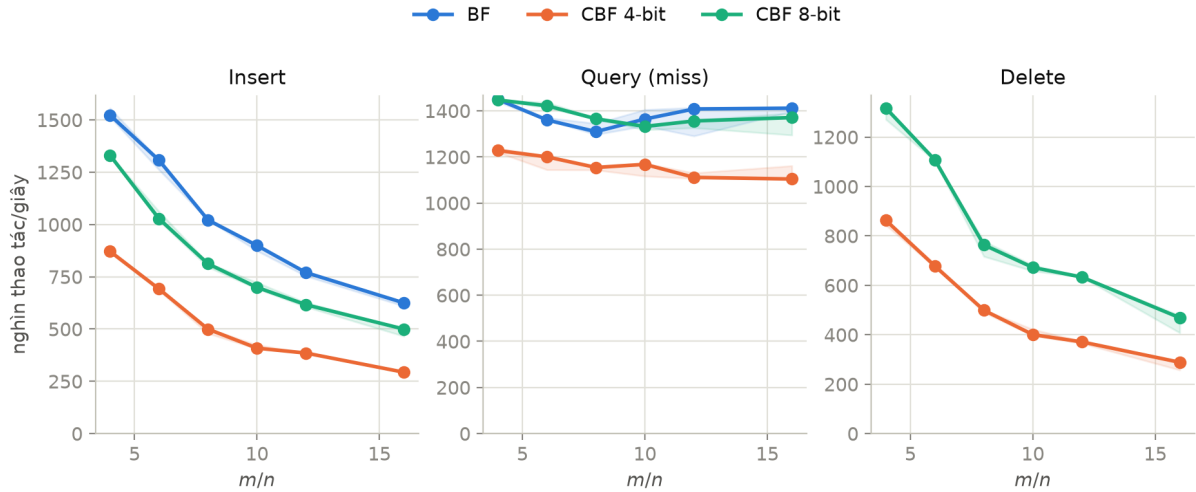
m/n	m (ô)	BF (byte)	CBF 4-bit (byte)	CBF 8-bit (byte)	CBF4/BF
4	400 000	50 000	200 000	400 000	$4.0\times$
6	600 000	75 000	300 000	600 000	$4.0\times$
8	800 000	100 000	400 000	800 000	$4.0\times$
10	1 000 000	125 000	500 000	1 000 000	$4.0\times$
12	1 200 000	150 000	600 000	1 200 000	$4.0\times$
16	1 600 000	200 000	800 000	1 600 000	$4.0\times$

6.4 Thông lượng (H3)

Bảng 4: E1: Median throughput qua 5 lượt (nghìn thao tác/giây; CBF4 = CBF 4-bit)

m/n	k	Insert		Query (miss)		Query (hit) / Delete	
		BF	CBF4	BF	CBF4	BF hit	CBF4 del
4	3	1 523	871	1 449	1 228	1 211	863
6	4	1 307	692	1 360	1 200	1 054	678
8	6	1 021	500	1 309	1 154	815	499
10	7	900	409	1 364	1 168	723	401
12	8	770	385	1 408	1 111	654	371
16	11	625	293	1 411	1 104	543	288

Tại $m/n = 10$ (Bảng 4, Hình 3): BF đạt 900 nghìn insert/s so với 409 của CBF 4-bit (nhANH gấp 2.20 lần) vì CBF phải đọc-sửa-ghi nibble thay vì chỉ bật bit. Với query trên khóa vắng mặt, BF đạt 1 364 nghìn thao tác/s so với 1 168 của CBF (gấp 1.17 lần). Query trên khóa vắng mặt (miss) nhanh hơn trên khóa có mặt (hit): với BF là 1 364 so với 723 nghìn thao tác/s, vì truy vấn miss thoát sớm ngay khi gặp bit 0 đầu tiên, còn truy vấn hit luôn phải thăm đủ k vị trí. Delete chuẩn của CBF 4-bit đạt 401 nghìn thao tác/s; phép đo giả định caller biết khóa đã chèn và không cộng thêm một lượt Query. Mỗi điểm là median của 5 lượt, dải tô trên Hình 3 là IQR. Các số và tỉ số phản ánh bản cài đặt cụ thể dùng `bitarray/bytearray`, thao tác nibble và vòng lặp Python; chúng không chứng minh mọi cài đặt BF luôn nhanh hơn mọi cài đặt CBF.



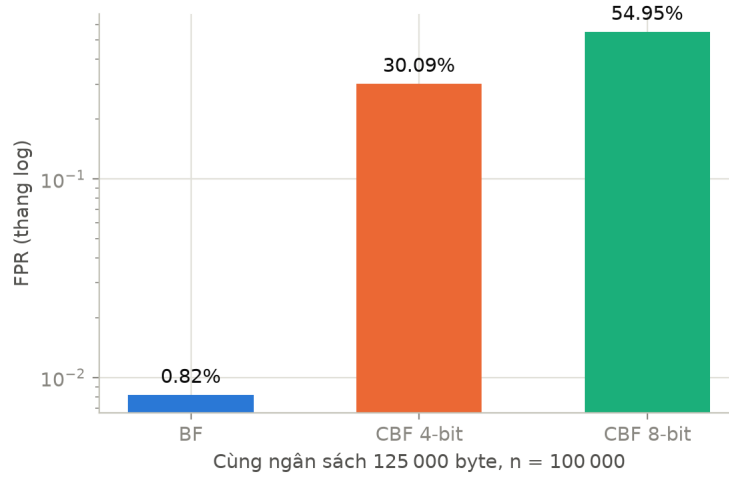
Hình 3: Median throughput insert, query (miss) và delete theo m/n ; dải tô là IQR của 5 lượt đo.

6.5 Đánh đổi FPR khi cố định payload bộ nhớ

Khi dung lượng payload là ràng buộc cứng (ví dụ SRAM on-chip), phép so này cho thấy chi phí FPR của counter; nó không làm hai cấu trúc tương đương chức năng vì BF không hỗ trợ delete. Với cùng 125 000 byte (Bảng 5, Hình 4): BF có 1 000 000 ô nên FPR chỉ 0.82%; CBF 4-bit còn 250 000 ô, FPR vọt lên 30.1%; CBF 8-bit còn 125 000 ô, FPR 54.9%. Khoảng cách 37–67 lần (từ hơn một tới gần hai bậc độ lớn) là đánh đổi dung lượng–FPR để có counter và khả năng xóa.

Bảng 5: E3: Cùng payload 125 000 byte, $n = 100\,000$, k tối ưu cho từng cấu trúc

Cấu trúc	Số ô	bit/ô	k	FPR lý thuyết	FPR thực nghiệm
BF	1 000 000	1	7	0.819%	0.819%
CBF 4-bit	250 000	4	2	30.3%	30.1%
CBF 8-bit	125 000	8	1	55.1%	54.9%



Hình 4: FPR (thang log) khi cả ba mảng backing dùng cùng payload byte.

6.6 Contract của delete và stress test xóa sai

E4 (Bảng 6) trước hết kiểm tra trường hợp dùng đúng: tại $m/n = 8$, counter lớn nhất sau insert chỉ là 8 nên không có overflow; sau khi xóa đúng 20 000 khóa, cả 80 000 khóa thật còn lại vẫn được Query nhận ra (0 false negative).

Phần hai dùng một bộ lọc riêng đã chèn đủ $n = 100\,000$ khóa (chưa xóa khóa nào) và cố ý vi phạm contract của delete: hệ thống nhằm câu trả lời “có thể có” là bằng chứng membership và thử xóa 2 164 khóa chưa từng được chèn. Dù dùng Query guard, 2 041 yêu cầu vẫn lọt qua vì chính các khóa này là false positive. Các lần giảm nhằm counter làm 8 063 phần tử thật (8.06% của n) biến mất khỏi bộ lọc. Đây là kết quả có điều kiện của stress test với hàng nghìn yêu cầu xóa sai, *không phải* tỉ lệ false negative mặc định của CBF và không được tổng quát hóa sang workload dùng đúng. Biện pháp chính là chỉ xóa khóa được xác nhận từ nguồn dữ liệu chuẩn; rebuild giúp khôi phục nếu contract hoặc thông tin counter đã bị phá vỡ.

Bảng 6: E4: Xóa đúng và stress test xóa sai trong CBF

Dại lượng	Giá trị
Cấu hình	$m/n = 8, k = 6, n = 100\,000$
Counter lớn nhất sau insert	8
Xóa đúng / FN trong khóa thật còn lại	20 000 / 0
Khóa vắng mặt bị báo nhầm “có” (false positive)	2 164
Lần xóa sai lọt qua query guard	2 041
Phần tử thật bị mất trong stress test	8 063
Tỉ lệ false negative trong stress test	8.063%

7 Các biến thể liên quan

Blocked Bloom Filter [4]: gói toàn bộ k bit của một khóa vào một block bằng đúng một cache line, giảm số cache miss mỗi truy vấn xuống ~ 1 , đổi lấy FPR tăng nhẹ; RocksDB dùng biến thể cache-line-local trong full filter [9]; không hỗ trợ xóa. **Cuckoo Filter** [7]: lưu fingerprint trong bảng băm cuckoo, *hỗ trợ xóa*, và cần ít không gian hơn cả BF tối ưu khi FPR mục tiêu $\varepsilon < 3\%$; nhược điểm là chèn có thể thất bại khi hệ số tải vượt $\sim 95\%$. **d-left CBF** (dlCBF) [6]: dùng d-left hashing + fingerprint, tiết kiệm khoảng một nửa không gian so với CBF 4-bit. Đây là một lựa chọn thay thế CBF tốt trên phần cứng.

8 Khuyến nghị lựa chọn

1. Tập tính hoặc rebuild được:

(SSTable bất biến, Cache Digest, chữ ký DPI ít đổi) dùng **BF**.

Chọn $m = -n \ln p / (\ln 2)^2$ và $k = \text{round}((m/n) \ln 2)$ theo FPR mục tiêu p (ví dụ $p = 1\% \Rightarrow m/n \approx 9.6, k = 7$).

2. Cần xóa động hoặc đếm bội:

dùng **CBF 4-bit** khi hệ thống có nguồn dữ liệu chuẩn để bảo đảm chỉ xóa khóa đã chèn. Theo dõi overflow; rebuild từ nguồn dữ liệu chuẩn khi counter mất thông tin hoặc contract delete có thể đã bị vi phạm.

3. Cần xóa và bộ nhớ/FPR là ràng buộc chặt: cân nhắc **Cuckoo Filter**, đặc biệt khi FPR mục tiêu $< 3\%$, hoặc **dlCBF** trên phần cứng; theo dõi hệ số tải và thất bại insert.

4. Nghẽn vì cache miss: chuyển sang **Blocked Bloom Filter**, chấp nhận FPR tăng nhẹ.

9 Kết luận

Thực nghiệm xác nhận bốn giả thuyết: FPR của BF/CBF trùng nhau và bám công thức lý thuyết trong sai số lấy mẫu (H1), payload CBF 4-bit bằng $4 \times$ BF (H2), và trong cài đặt Python đang xét BF có median throughput cao hơn CBF 4-bit ở các thao tác chung (H3); FPR giảm mũ theo m/n với cực tiểu theo k tại k^* (H4). Bức tranh tổng thể: *CBF mua khả năng xóa bằng payload counter lớn hơn và một contract delete chặt hơn*. Khi chỉ xóa khóa chắc chắn đã chèn và không overflow, thí nghiệm không quan sát false negative; false negative xuất hiện trong stress test khi cố ý xóa các khóa vắng mặt. Do đó BF phù hợp với tập tính/rebuild được; CBF phù hợp với tập động khi có nguồn dữ liệu chuẩn cho delete; và Cuckoo Filter đáng khảo sát khi cần xóa với yêu cầu FPR/bộ nhớ chặt.

Hạn chế. Benchmark dùng một bản cài đặt Python cụ thể; dù đã báo median/IQR qua 5 lượt, cả số tuyệt đối lẫn tỉ số không đại diện cho mọi bản cài đặt C/C++/phần cứng. FPR dùng một tập khóa tắt định và một cặp seed; KTC Wilson chỉ phản ánh sai số trên các truy vấn vắng mặt, chưa bao phủ biến thiên qua nhiều họ hash/seed. Công thức (2) là xấp xỉ (và là cận dưới); với m nhỏ nên đo thực nghiệm. Phân tích giả định hàm băm lý tưởng; double hashing chỉ hợp lệ tiệm cận và môi trường đối kháng cần hàm băm có khóa. Các số bộ nhớ chỉ là payload của mảng backing, không phải RSS.

Tài liệu

- [1] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [2] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, “Summary Cache: A Scalable Wide-Area Web Cache Sharing Protocol,” *IEEE/ACM Transactions on Networking*, vol. 8, no. 3, pp. 281–293, 2000.
- [3] A. Broder and M. Mitzenmacher, “Network Applications of Bloom Filters: A Survey,” *Internet Mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [4] F. Putze, P. Sanders, and J. Singler, “Cache-, hash-, and space-efficient Bloom filters,” *ACM Journal of Experimental Algorithmics*, vol. 14, 2009.
- [5] A. Kirsch and M. Mitzenmacher, “Less hashing, same performance: Building a better Bloom filter,” *Random Structures & Algorithms*, vol. 33, no. 2, pp. 187–218, 2008.

- [6] F. Bonomi, M. Mitzenmacher, R. Panigrahy, S. Singh, and G. Varghese, “An Improved Construction for Counting Bloom Filters,” in *Algorithms: ESA 2006*, LNCS 4168, pp. 684–695.
- [7] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, “Cuckoo Filter: Practically Better Than Bloom,” in *Proc. 10th ACM CoNEXT*, pp. 75–88, 2014.
- [8] S. Dharmapurikar, P. Krishnamurthy, T. Sproull, and J. Lockwood, “Deep Packet Inspection using Parallel Bloom Filters,” in *IEEE Hot Interconnects 11*, 2003.
- [9] RocksDB, “RocksDB Bloom Filter,” project documentation, accessed Aug. 10, 2026.